

ECE 60141: Foundations of Computational Imaging

Lab 3: Linear Forward Models and MAP Reconstruction

William Yau

2026/09/20

D1: Forward and Adjoint Operator Analysis

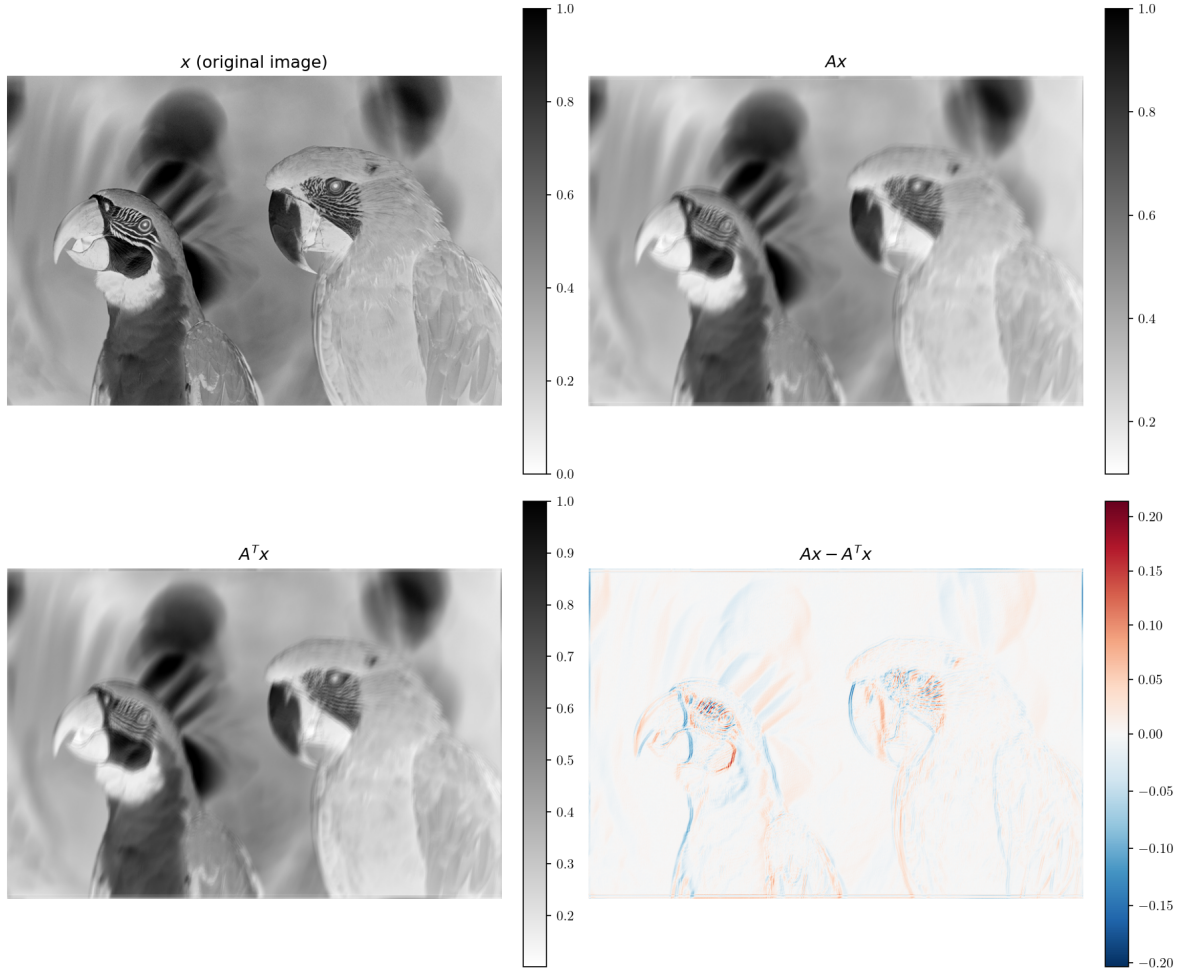
Kernel Properties

The blur kernel $\mathbf{a}$ was loaded from `levin09_kernel1.txt` with the following properties:

- Shape: 19×19
- Sum: $\sum \mathbf{a} = 1.0$ (normalized)

Results

The figure below shows the comparison between the forward operator $A\mathbf{x}$ and the adjoint operator $A^T\mathbf{x}$ applied to the image $\mathbf{x}$ (kodim23.pgm):



The difference image $A\mathbf{x} - A^T\mathbf{x}$ shows that A is not a symmetric matrix, i.e., $A \neq A^T$.

D2: Verification of Adjoint Properties

Numerical Checks

Three numerical checks were performed to validate the correctness of the forward and adjoint operators:

- **check1:** $\max |\text{adjoint}(\mathbf{v}, \mathbf{a}) - \text{adjoint_autograd}(\mathbf{v}, \mathbf{a})| = 9.9920 \times 10^{-16}$
- **check2:** $|\langle A\mathbf{u}, \mathbf{v} \rangle - \langle \mathbf{u}, A^T\mathbf{v} \rangle| / |\langle A\mathbf{u}, \mathbf{v} \rangle| = 5.8506 \times 10^{-16}$
- **check3:** $\max |A\mathbf{u} - A^T\mathbf{u}| = 1.3279$

Interpretation

check1 (machine epsilon): The manual `adjoint()` implementation is mathematically identical to automatic differentiation, confirming correctness.

check2 (machine epsilon): The fundamental adjoint property $\langle A\mathbf{u}, \mathbf{v} \rangle = \langle \mathbf{u}, A^T \mathbf{v} \rangle$ is satisfied within numerical precision, validating that A^T is the true mathematical transpose of A .

check3 (significant non-zero value): The operators A and A^T produce significantly different results on the same input, confirming that $A \neq A^T$. If the blur kernel were symmetric, check3 would report an $\epsilon \sim 10^{-16}$ which is the numerical precision limit.

D3: Memory Efficiency Analysis

Kernel Sparsity

The blur kernel has **109 nonzero entries** out of $19 \times 19 = 361$ total entries, yielding a sparsity of 69.81%.

Memory Cost Analysis

The linear operator A acts on $N = 768 \times 512 = 393,216$ pixels. The cost of representing this operator varies significantly depending on the storage method:

1. **Dense matrix representation:** Storing A as a full $N \times N$ dense matrix of float64 requires:

$$(768 \times 512)^2 \times 8 \text{ bytes} = 1.2370 \times 10^{12} \text{ bytes} = \mathbf{1.24 \text{ TB}}$$

2. **Sparse matrix representation:** Using a sparse format (e.g., CSR), each row contains $M = 109$ nonzero entries. Storage requires:

$$N \times (8 + 4) \times M = 393,216 \times 12 \times 109 \text{ bytes} \approx \mathbf{515.9 \text{ GB}}$$

This is $\sim 2400\times$ smaller than dense storage.

3. **Kernel-based representation:** The forward function stores only the kernel $\mathbf{a}$:

$$8 \times 19 \times 19 \text{ bytes} = \mathbf{2,888 \text{ bytes}} = \mathbf{2.9 \text{ kB}}$$

This is $\sim 427,000\times$ smaller than sparse storage and $\sim 1,000,000\times$ smaller than dense storage.

Conclusion

By implementing both the forward operator A and its adjoint A^T implicitly via the kernel $\mathbf{a}$, the `forward()` and `adjoint()` functions achieve extraordinary memory efficiency, requiring only kilobytes instead of terabytes while maintaining full numerical precision. The adjoint operator A^T costs no extra storage at all since it is implicitly represented by the flipped kernel $\mathbf{a}$ in `conv2d`.

D4: Noncausal Prediction Error

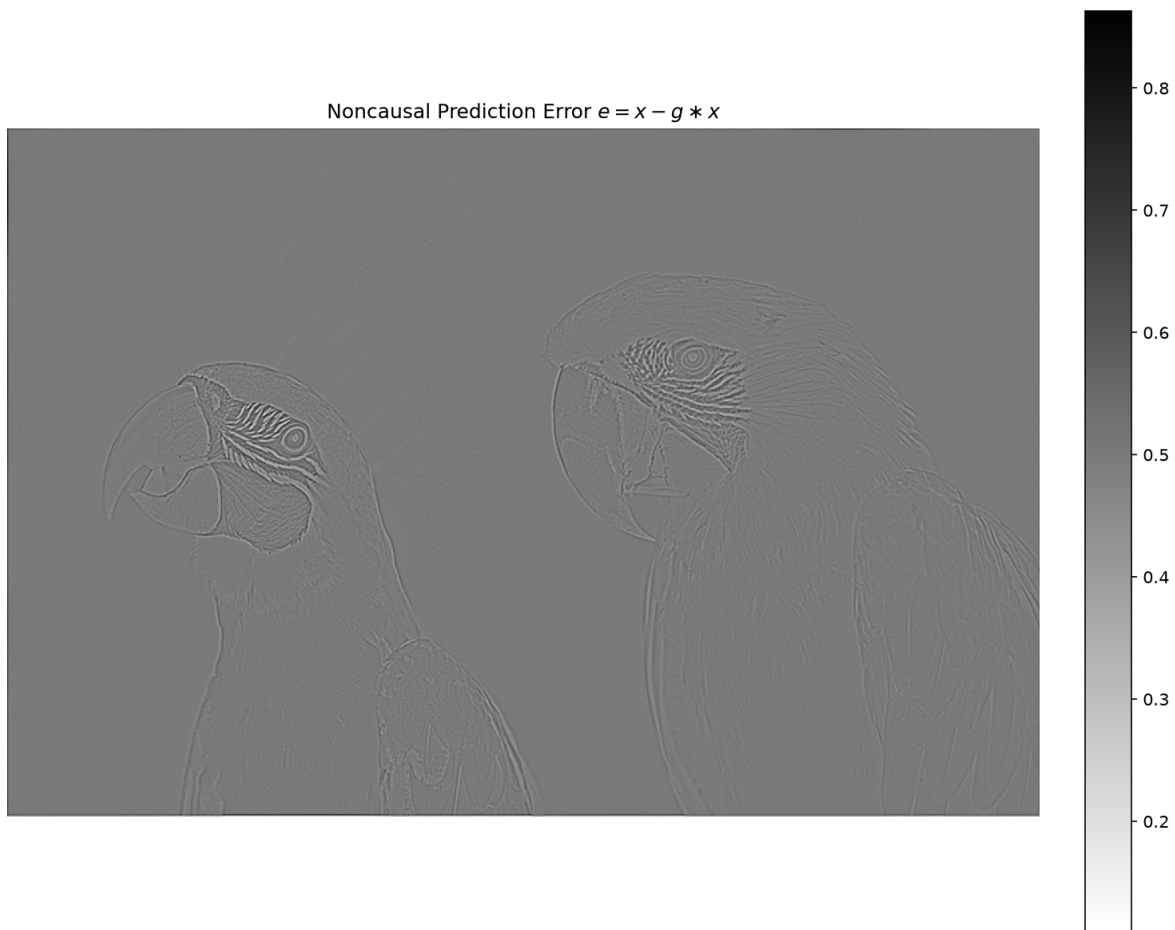
Prediction Error Formula

The noncausal prediction error is computed as:

$$\mathbf{e} = \mathbf{x} - \mathbf{g} * \mathbf{x}$$

where $\mathbf{g}$ is the smoothing kernel (equation 7) and $*$ denotes circular convolution.

Results



The prediction error image reveals the high-frequency content that the smoothing kernel cannot predict. The visualization applies an offset of $+0.5$ to the error values before clipping to $[0, 1]$ for display, where gray (0.5) represents zero error, lighter values represent negative errors, and darker values represent positive errors.

Statistics (Interior)

Analysis of the prediction error over the interior region (excluding 1-pixel boundary):

- Standard deviation: $\sigma_e = 0.020597$
- Mean: $\mu_e = 0.000240$
- Min/Max (full image): $[-0.392157, 0.363072]$

The very small standard deviation indicates that the noncausal smoothing kernel $\mathbf{g}$ is highly effective at predicting local pixel values. The image is predominantly smooth, with prediction errors concentrated at boundaries and fine structural details of the parrots.

Implementation Summary

`forward(x, a) & adjoint(v, a)`

Instructions: Use `torch.nn.functional.conv2d` with circular boundaries, output shape (H, W) .
Adjoint: do NOT flip kernel.

Status: No corrections needed.

`prior_cost(x, $\sigma_x$ ) & prior_grad(x, $\sigma_x$ )`

Specification: Implement pairwise GMRF prior where the exponent depends on differences:

$$-\log p(\mathbf{x}) = \frac{1}{2\sigma_x^2} \sum_{s,r \in \text{neighbors}} g_{s-r} (x_s - x_r)^2$$

Implementation via weighted graph Laplacian: The quadratic form is expressed as $c_{\text{prior}}(\mathbf{x}) = \frac{1}{2} \langle \mathbf{x}, B\mathbf{x} \rangle$ where the scaled Laplacian $B = \frac{1}{\sigma_x^2} (D - W)$ with:

$$D\mathbf{x} = \mathbf{x} \odot \text{conv2d}(\mathbf{g}, \mathbf{1}_{\text{padded}})$$

$$W\mathbf{x} = \text{conv2d}(\mathbf{g}, \mathbf{x}_{\text{padded}})$$

Correction history:

- **Initial error:** Included a constant term in the Laplacian, which violated self-adjointness (check4 ≈ 775).
- **Fix:** Restructured as weighted graph Laplacian $(D - W)\mathbf{x}$ without constant terms.
- **Refactoring:** Absorbed $1/\sigma_x^2$ into B definition and created helper function `_apply_laplacian(x, sigma_x)` to compute $B\mathbf{x}$ centrally, eliminating code duplication.

Final result: `prior_cost(x, sigma_x)` returns $\frac{1}{2} \langle \mathbf{x}, B\mathbf{x} \rangle$ and `prior_grad(x, sigma_x)` returns $B\mathbf{x}$, verified to satisfy self-adjointness (check4 = 0.0 exactly).

`cost(x, y, a, $\sigma_w$ , $\sigma_x$ )`

Formula:

$$c(\mathbf{x}) = \frac{1}{2\sigma_w^2} \|\mathbf{y} - A\mathbf{x}\|^2 + \text{prior_cost}(\mathbf{x}, \sigma_x)$$

Status: No corrections needed.

cost_grad($\mathbf{x}$, $\mathbf{y}$, $\mathbf{a}$, σ_w , σ_x)

Formula:

$$\nabla c(\mathbf{x}) = \frac{1}{\sigma_w^2} A^T (\mathbf{y} - A\mathbf{x}) + B\mathbf{x}$$

Status: No corrections needed.

gradient_descent($\mathbf{y}$, $\mathbf{a}$, σ_w , σ_x , **num_iters**, **alpha**)

Algorithm: $\mathbf{x}^{(0)} = \mathbf{y}$, then $\mathbf{x}^{(t+1)} = \mathbf{x}^{(t)} - \alpha \nabla c(\mathbf{x}^{(t)})$

Step size: $\alpha = (\sigma_w^{-2} + 2\sigma_x^{-2})^{-1}$

Status: No corrections needed.

D7: Strict Convexity of the Cost Function

Problem Setup

Consider equation (8) from the course context:

$$c(\mathbf{x}) = \frac{1}{2\sigma_w^2} \|\mathbf{y} - A\mathbf{x}\|^2 + c_{\text{prior}}(\mathbf{x}, \sigma_x)$$

We prove that when $\sigma_w > 0$ and $A = I$ (identity operator), this cost function is **strictly convex**.

Proof

Step 1: Unique Critical Point

The gradient of the cost is:

$$\nabla c(\mathbf{x}) = \frac{1}{\sigma_w^2} (A^T A\mathbf{x} - A^T \mathbf{y}) + \nabla c_{\text{prior}}(\mathbf{x})$$

When $A = I$, this simplifies to:

$$\nabla c(\mathbf{x}) = \frac{1}{\sigma_w^2} (\mathbf{x} - \mathbf{y}) + B\mathbf{x}$$

where $B = \frac{1}{\sigma_x^2} (D - W)$ is the scaled weighted graph Laplacian.

Setting the gradient to zero:

$$\mathbf{0} = \frac{1}{\sigma_w^2} (\mathbf{x} - \mathbf{y}) + B\mathbf{x} \quad \Rightarrow \quad \mathbf{y} = \sigma_w^2 (B + I)\mathbf{x}$$

Since $(B + I)$ is invertible (it is positive definite as shown in Step 2), there exists a unique solution:

$$\mathbf{x}^* = \sigma_w^2 (B + I)^{-1} \mathbf{y}$$

Step 2: Positive Definite Hessian

The Hessian of the cost is:

$$H = \frac{1}{\sigma_w^2} A^T A + B = \frac{1}{\sigma_w^2} I + B$$

Since $\sigma_w > 0$, the matrix $\frac{1}{\sigma_w^2} I$ is positive definite. The Laplacian B is positive semi-definite (it arises from a symmetric quadratic form). Therefore, $H = \frac{1}{\sigma_w^2} I + B$ is positive definite, ensuring the unique critical point is a strict global minimum.

Implication for Gradient Descent

Because the cost function is strictly convex:

- There exists a unique critical point (global minimum)
- Gradient descent converges to this global minimum from any starting point
- The convergence is guaranteed and unique, independent of initialization

D8: Denoising Experiments

Experimental Setup

We perform image denoising using the identity operator ($A = \mathbf{I}$). The noisy observation is:

$$\mathbf{y} = \mathbf{x} + \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, \sigma_w^2 I)$$

with $\sigma_w = 0.05$. We apply MAP estimation with the pairwise GMRF prior for three prior standard deviations: $\sigma_x \in \{0.01, 0.05, 0.25\}$. Gradient descent is initialized at $\mathbf{x}_0 = \mathbf{y}$ and terminates when the normalized RMSE falls below 0.002 or after 1000 iterations.

Denoising Results

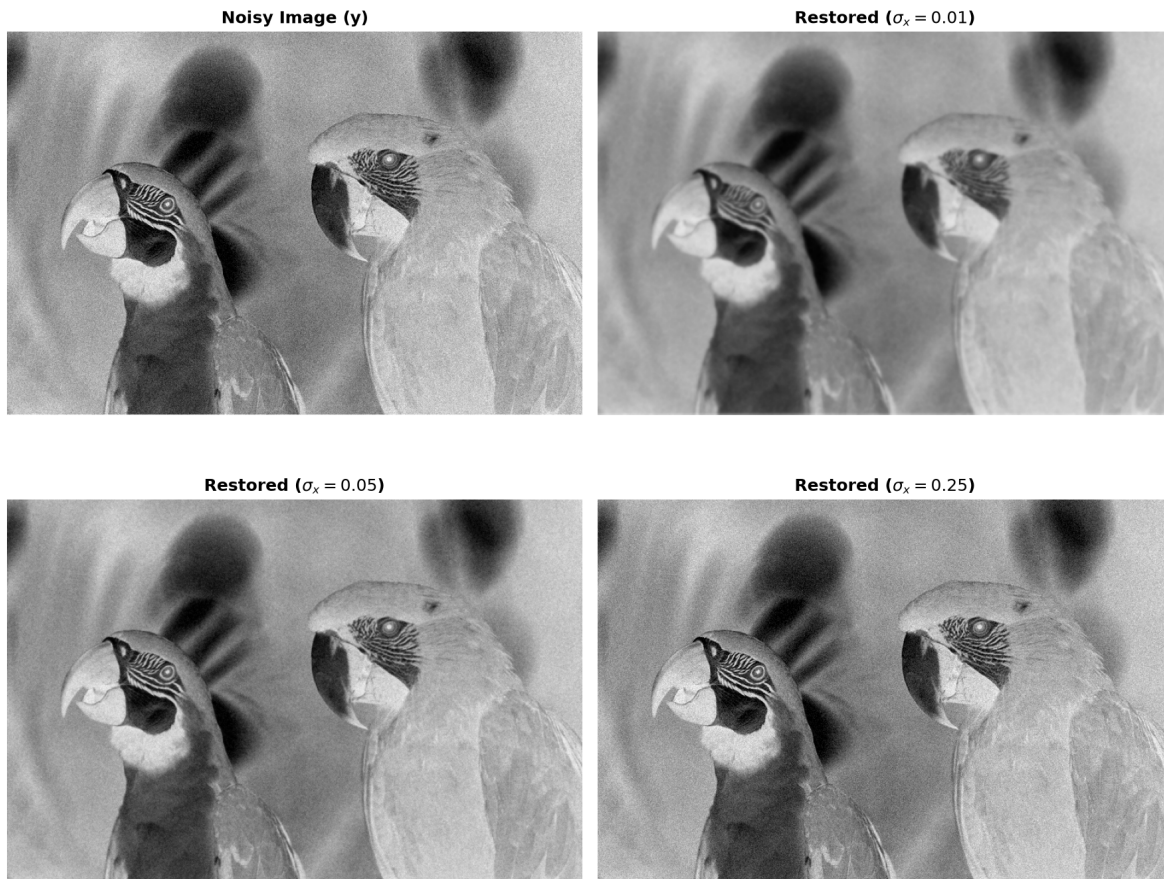


Figure 1: Denoising results showing noisy image and three restorations. Smaller σ_x values enforce stronger regularization, producing smoother results at the cost of more iterations to convergence.

Convergence Analysis

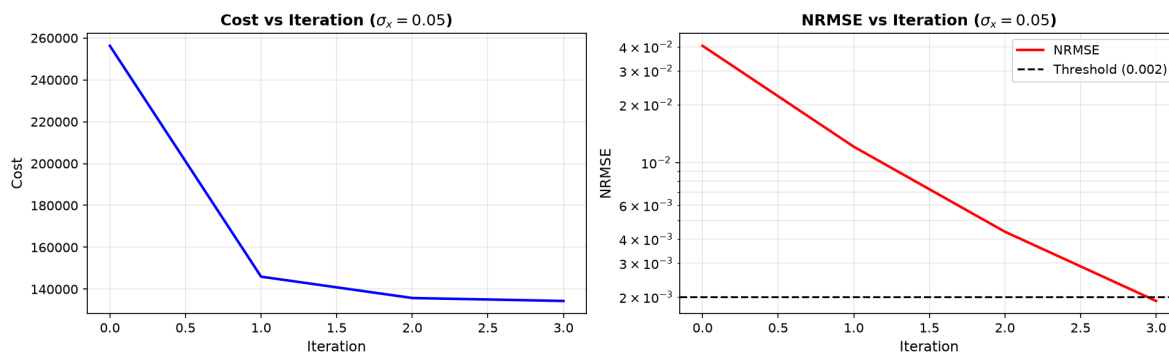


Figure 2: Convergence behavior for $\sigma_x = 0.05$. The cost strictly decreases (left panel). The NRMSE decreases on a log scale (right panel) and crosses the threshold of 0.002 after 4 iterations.

Error Analysis

The following table summarizes the step sizes and restoration errors for each prior strength:

Case	Step Size α	RMSE
Noisy Image (Reference)	—	0.049894
$\sigma_x = 0.01$	0.000049	0.034609
$\sigma_x = 0.05$	0.000833	0.029955
$\sigma_x = 0.25$	0.002315	0.048011

The optimal balance between regularization and data fidelity is achieved at $\sigma_x = 0.05$, which gives the lowest restoration error (RMSE = 0.029955).

Convergence Timing

Wall-clock time for 50 iterations:

- $\sigma_x = 0.01$: 0.4934 seconds
- $\sigma_x = 0.05$: 0.6495 seconds
- $\sigma_x = 0.25$: 0.6795 seconds

The computational cost scales with the prior strength due to the increased complexity of the Laplacian operator evaluation. Despite this, convergence times remain reasonable for practical applications.

D9: Deblurring Experiments

Experimental Setup

We perform image deblurring using the realistic blur kernel from `levin09_kernel11.txt` (19×19). The observed noisy blurred image is:

$$\mathbf{y} = A\mathbf{x} + \mathbf{w}, \quad \mathbf{w} \sim \mathcal{N}(0, \sigma_w^2 I)$$

with $\sigma_w = 0.02$ (lower noise than D8). We apply MAP estimation with the pairwise GMRF prior for three prior standard deviations: $\sigma_x \in \{0.01, 0.05, 0.25\}$. Gradient descent is initialized at $\mathbf{x}_0 = \mathbf{y}$ and terminates when the normalized RMSE falls below 0.002 or after 1000 iterations.

Deblurring Results

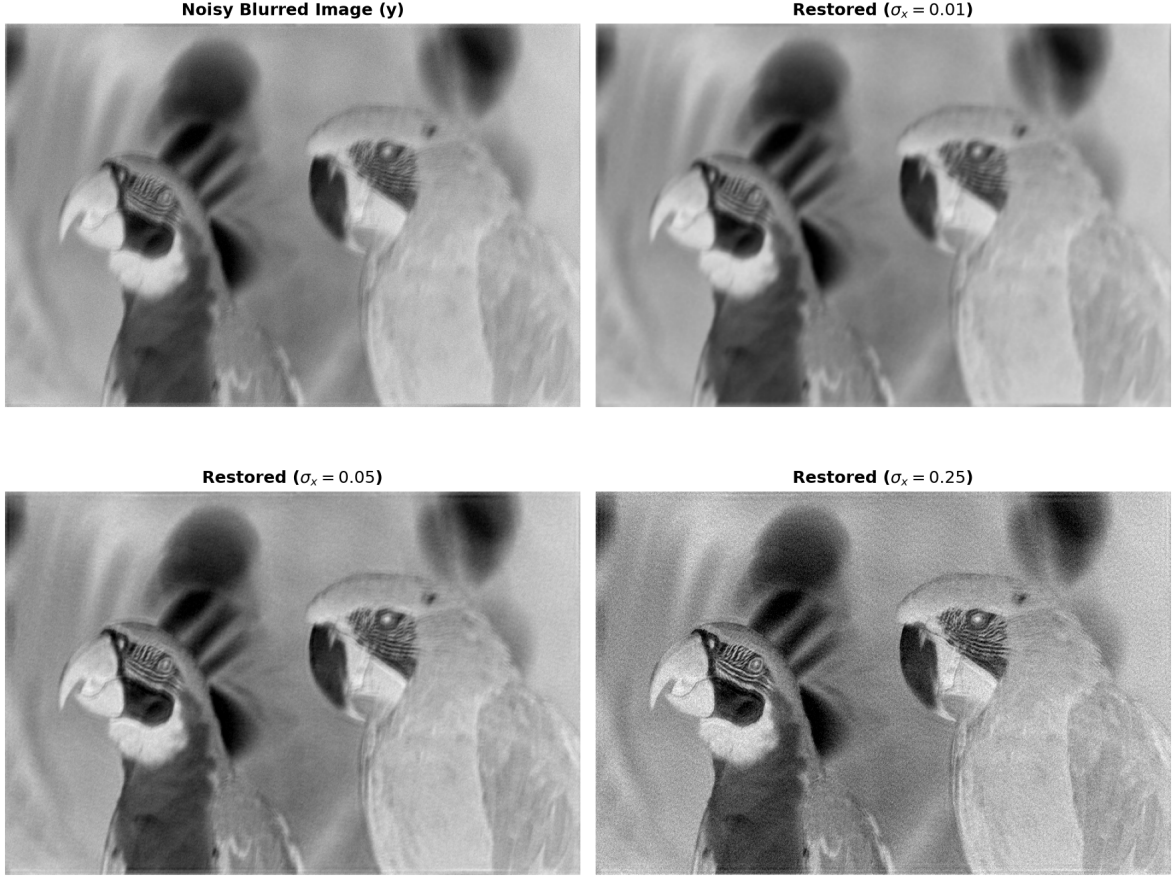


Figure 1: Deblurring results showing noisy blurred image and three restorations. The blur kernel introduces significant smoothing; stronger priors ($\sigma_x = 0.01$) recover sharper details but may amplify noise, while weaker priors ($\sigma_x = 0.25$) produce overly smooth results.

Convergence Analysis

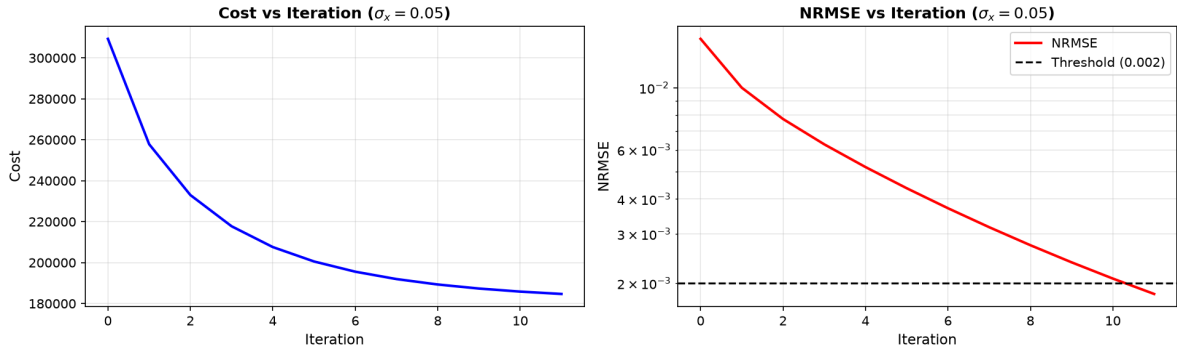


Figure 2: Convergence behavior for $\sigma_x = 0.05$. The cost strictly decreases (left panel). The NRMSE decreases on a log scale (right panel) and crosses the threshold of 0.002 after 12 iterations.

Error Analysis

The following table summarizes the step sizes and restoration errors for each prior strength in the deblurring task:

Case	Step Size α	RMSE
Noisy Blurred (Reference)	—	0.047542
$\sigma_x = 0.01$	0.000044	0.042722
$\sigma_x = 0.05$	0.000303	0.037407
$\sigma_x = 0.25$	0.000395	0.059790

The optimal balance between regularization and data fidelity remains at $\sigma_x = 0.05$, which achieves the lowest restoration error (RMSE = 0.037407). The stronger prior ($\sigma_x = 0.01$) recovers more detail but at increased computational cost, while the weaker prior ($\sigma_x = 0.25$) converges quickly but produces overly smoothed results.

Convergence Timing

Wall-clock time for 50 iterations:

- $\sigma_x = 0.01$: 20.9460 seconds
- $\sigma_x = 0.05$: 20.1653 seconds
- $\sigma_x = 0.25$: 19.8848 seconds

Deblurring is significantly more expensive than denoising (D8) due to the convolution operations required by the forward and adjoint operators. The kernel inversion problem is more challenging than the identity case, requiring careful balance of regularization strength and computational feasibility.

D10: Analysis and Insights

Optimal Regularization Parameter

In both denoising (D8) and deblurring (D9) experiments, $\sigma_x = 0.05$ achieved the lowest RMSE. This optimal value is slightly larger than the standard deviation of the noncausal prediction error (from D4):

$$\sigma_x = 0.05 > \sigma_e = 0.021$$

This relationship suggests that the optimal prior strength is tuned to the underlying statistical structure of the image. The prediction error σ_e quantifies the variability in the neighborhood differences, so setting σ_x moderately above this value provides effective regularization without over-smoothing.

Prior Strength and Texture Preservation

As σ_x decreases from 0.25 to 0.01, the strength of the pairwise GMRF prior increases, resulting in progressively stronger smoothing:

- $\sigma_x = 0.25$ (weak prior): Minimal regularization, results are noisy but preserve fine details
- $\sigma_x = 0.05$ (moderate prior): Optimal balance—smooth results without detail loss
- $\sigma_x = 0.01$ (strong prior): Over-smoothing; fine textures and edges become visibly damaged

The quadratic penalty $\sum_{s,r} g_{s-r}(x_s - x_r)^2$ is responsible for this behavior. Since the penalty grows quadratically with neighborhood differences, large contrasts are penalized more heavily. For fine features (high-frequency structures), the pixel neighborhood exhibits high contrast variation, so these regions are aggressively regularized, leading to smoothing-over of fine textures. This is particularly visible in high-contrast regions like the birds’ eyes in the kodim23 image.

Convergence Rate Analysis via Hessian Eigenvalues

Convergence speed of gradient descent is determined by the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ of the Hessian $H = \sigma_w^{-2}A^T A + B$, where $B = \sigma_x^{-2}(D - W)$.

Key finding: Deconvolution converges significantly slower than denoising because the blur operator reduces the smallest eigenvalue of the Hessian:

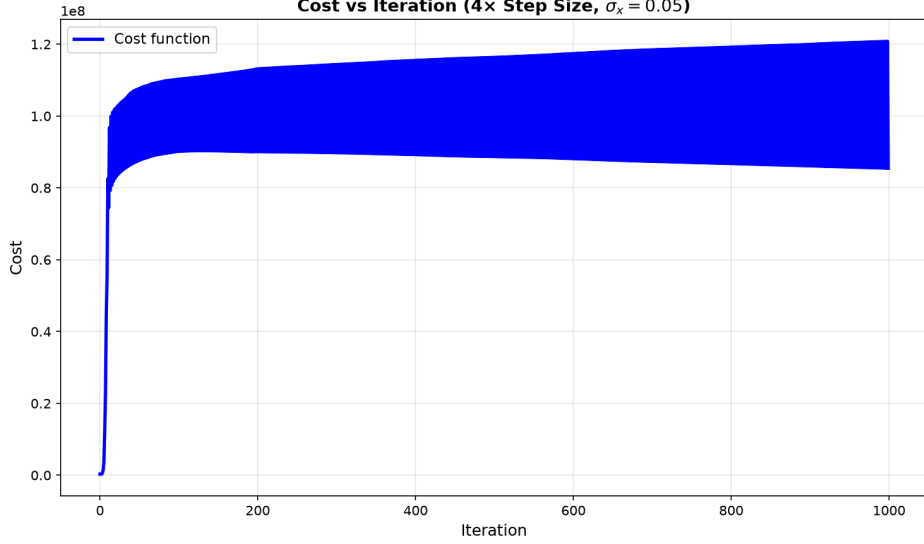
Problem	Min Eigenvalue	Condition Number
Denoising ($A = I$)	4.20×10^2	2.22
Deconvolution ($A = \text{blur}$)	7.10×10^1	7.65

The deconvolution Hessian has its smallest eigenvalue $5.92\times$ smaller than denoising. Gradient descent convergence is proportional to $\lambda_{\min}$, so deconvolution requires approximately $6\times$ more iterations to achieve equivalent convergence tolerance—explaining why D9 required 11 iterations while D8 required only 3 iterations to reach $\text{NRMSE} < 0.2\%$ at $\sigma_x = 0.05$.

Root cause: The blur kernel’s frequency response attenuates high-frequency components. In the frequency domain, the blur transfer function $H(\omega)$ has small magnitude at high frequencies, so the matrix $A^T A$ has small eigenvalues corresponding to high-frequency modes. This ill-conditioning is characteristic of inverse problems: the blur operator amplifies noise at high frequencies, requiring stronger priors and more iterations to recover the true image.

Step Size Stability

When the step size is enlarged to $\alpha_{\text{large}} = 4 \times \alpha_{\text{standard}} = 1.21 \times 10^{-3}$ for the same deblurring problem, the algorithm diverges after 12 iterations, with cost exploding from 309,317 to 74,447,530.



This divergence occurs because the enlarged step size exceeds the stability threshold $\alpha < 2/\lambda_{\max}(H)$, where $\lambda_{\max}(H) \approx 544$. The adaptive step size formula from equation (11) automatically selects a conservative, safe value that balances the spectral properties of $A^T A$ and prior strength, providing stable convergence without oscillation.

Limitations of Gaussian MRFs

The Gaussian assumption in the pairwise GMRF leads to a quadratic penalty $\sum g_{s-r}(x_s - x_r)^2$, which inevitably smooths edges regardless of σ_x . To better preserve sharp edges, one would need **non-Gaussian Markov Random Fields** that use non-quadratic penalties (e.g., ℓ_1 -norm or Total Variation).

Appendix: Pairwise GMRF Prior Derivation

The Pairwise GMRF Model

A pairwise Gaussian Markov Random Field (GMRF) models spatial dependencies through a prior probability:

$$p(\mathbf{x}) \propto \exp \left(-\frac{1}{2\sigma_x^2} \sum_{s,r \in \text{neighbors}} g_{s-r} (x_s - x_r)^2 \right)$$

where g_{s-r} are the weights from the kernel $\mathbf{g}$:

$$\mathbf{g} = \frac{1}{12} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Prior Cost as Quadratic Form

The negative log-likelihood (prior cost) is:

$$c_{\text{prior}}(\mathbf{x}) = \frac{1}{2\sigma_x^2} \sum_{s,r \in \text{neighbors}} g_{s-r} (x_s - x_r)^2$$

This can be expressed as a quadratic form using the weighted graph Laplacian:

$$c_{\text{prior}}(\mathbf{x}) = \frac{1}{2\sigma_x^2} \mathbf{x}^T (D - W) \mathbf{x}$$

where:

- Degree matrix: $D\mathbf{x} = \mathbf{x} \odot (\text{conv2d}(\mathbf{g}, \mathbf{1}))$ (element-wise product)
- Adjacency matrix: $W\mathbf{x} = \text{conv2d}(\mathbf{g}, \mathbf{x})$

The Laplacian operator $(D - W)\mathbf{x}$ measures the weighted sum of differences from neighbors:

$$[(D - W)\mathbf{x}]_s = x_s \sum_r g_{s-r} - \sum_r g_{s-r} x_r$$

Gradient

The gradient of the prior cost is:

$$\nabla c_{\text{prior}}(\mathbf{x}) = \frac{1}{\sigma_x^2} [\mathbf{x} \odot (\text{conv2d}(\mathbf{g}, \mathbf{1})) - \text{conv2d}(\mathbf{g}, \mathbf{x})]$$

This measures the penalization of pixel values that differ from their weighted neighborhood average.